

리눅스 5 - I/O

남궁명수

[1] I/O 란?

I/O는 Input/Output의 약자로, 프로세스가 자신의 CPU, 메모리 밖에 있는 대상과 데이터를 주고받는 작업이다.

Input

외부 대상 → 프로세스

Output

프로세스 → 외부 대상

대표적인 I/O 대상:

- 일반 파일과 디스크
- 네트워크 소켓
- 파이프와 FIFO
- 터미널과 장치 파일
- eventfd
- timerfd
- signalfd
- inotify

사용자 프로그램은 주로 다음 시스템 콜로 I/O를 요청한다.

```
read(fd, buffer, size);
write(fd, buffer, size);

recv(fd, buffer, size, flags);
send(fd, buffer, size, flags);
```

이 함수들은 모두 FD를 전달받는다.

따라서 리눅스 I/O를 이해하려면 먼저 FD가 무엇을 가리키는지 알아야 한다.

[2] 파일 디스크립터란?

파일 디스크립터(File Descriptor, FD)는 프로세스가 커널의 I/O 객체를 가리킬 때 사용하는 정수 번호다.

```
int fd = open("data.txt", O_RDONLY);
```

예를 들어, open()이 3을 반환했다고 해보자.

```
fd = 3
```

FD 3이라는 숫자 안에 파일 내용이 들어 있는 것은 아니다.

FD 3은 현재 프로세스의 FD TABLE에서 세 번째 항목을 찾기 위한 인덱스다.

```
FD 3
  ↓
프로세스의 FD Table
  ↓
struct file
  ↓
실제 파일 또는 커널 I/O 객체
```

소켓도 동일하게 FD로 표현된다.

```
FD 7
  ↓
FD Table
  ↓
struct file
  ↓
struct socket
  ↓
struct sock
  └─ 수신 버퍼
  └─ 송신 버퍼
  └─ 연결 상태
  └─ Wait Queue
```

따라서 select, poll, epoll은 정수 번호 자체를 감시하는 것이 아니다.

FD가 가리키는 커널 I/O 객체의 상태를 감시한다.

사용자 프로세스

fd = 3



프로세스의 FD Table

0	표준 입력
1	표준 출력
2	표준 오류
3	struct file 포인터

▼
struct file

- 파일 위치
- 상태 플래그
- 연산 함수
- 실제 커널 객체

[3] “I/O 준비됨”의 정확한 의미

select, poll, epoll은 I/O를 직접 수행하지 않는다.

지금 read() 또는 write()를 호출했을 때 오래 기다리지 않고 어느 정도 진행할 수 있는지를 알려준다.

select() 매뉴얼도 준비 상태를 “해당 I/O 연산을 블로킹 없이 수행할 수 있는 상태”로 정의한다.

[읽기 준비]

소켓이 읽기 준비 상태라는 것은 다음 중 하나 일 수 있다.

1. 수신 버퍼에 읽을 데이터가 있다.
2. 상대가 정상적으로 연결을 종료했다.
3. 연결 오류가 발생했다.
4. 리스닝 소켓의 Accept Queue에 새 연결이 있다.

상대가 정상적으로 연결을 종료한 경우에도 소켓은 읽기 준비 상태가 된다.
이때 read() 또는 recv()는 0을 반환할 수 있다.

따라서 다음과 같이 이해해야 한다.

EPOLLIN ≠ 정상 메시지 하나가 도착했다

EPOLLIN → read(), recv(), accept()가 블로킹 되지 않고 결과를 낼 수 있다

[쓰기 준비]

쓰기 준비 상태는 소켓의 송신 버퍼에 데이터를 넣을 공간이 있다는 뜻이다.

소켓 송신 버퍼에 여유가 생김

↓

EPOLLOUT

연결된 소켓은 평상시에 쓰기 가능한 경우가 많다.
따라서 항상 EPOLLOUT을 감시하면 이벤트 루프가 불필요하게 계속 깨어날 수 있다.
일반적으로 다음과 같이 처리한다.

보낼 데이터 없음

→ EPOLLOUT 감시하지 않음

send()가 일부 데이터만 전송

→ 남은 데이터를 사용자 공간에 보관

→ EPOLLOUT 감시 시작

EPOLLOUT 발생

→ 남은 데이터 다시 전송

전부 전송 완료

→ 전부 전송하면 EPOLLOUT 감시 해제

[4] Blocking I/O

Blocking I/O는 요청한 작업을 즉시 완료할 수 없으면 호출한 스레드를 대기시키는 방식이다.

```
ssize_t n = read(fd, buffer, size);
```

소켓 수신 버퍼가 비어 있다면 다음 순서로 진행된다.

1. read() 시스템 콜로 커널 모드에 진입한다.
2. 커널이 소켓 수신 버퍼를 확인한다.
3. 읽을 데이터가 없으면 현재 스레드를 소켓의 Wait Queue에 연결한다.
4. 스레드를 대기 상태로 바꾸고 CPU를 반납한다.
5. 데이터가 도착하면 스레드를 Runnable 상태로 깨운다.
6. 스케줄러가 스레드를 다시 실행한다.
7. read()가 데이터를 사용자 버퍼로 복사한다.
8. 사용자 모드로 돌아간다.

```
read()  
↓  
소켓 수신 버퍼 확인  
↓  
데이터 없음  
↓  
Wait Queue에 스레드 등록  
↓  
스레드 대기 및 CPU 반납
```

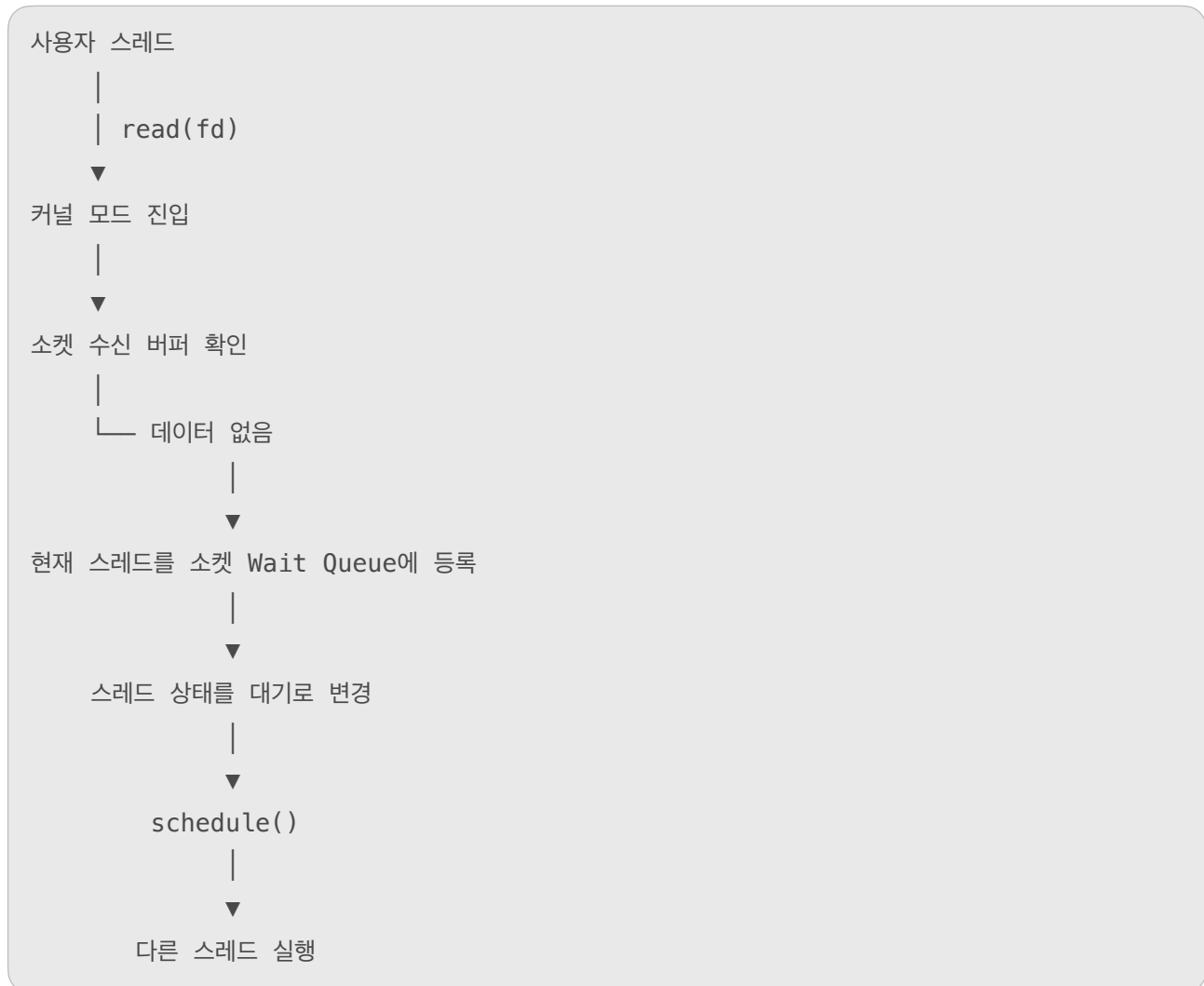
이후 패킷이 도착하면 다음 과정이 진행된다.

```
NIC에 패킷 도착  
↓  
네트워크 처리  
↓  
소켓 수신 버퍼에 데이터 저장  
↓  
Wait Queue 알림  
↓  
스레드 Runnable  
↓  
스케줄러가 스레드 실행  
↓  
read() 완료
```

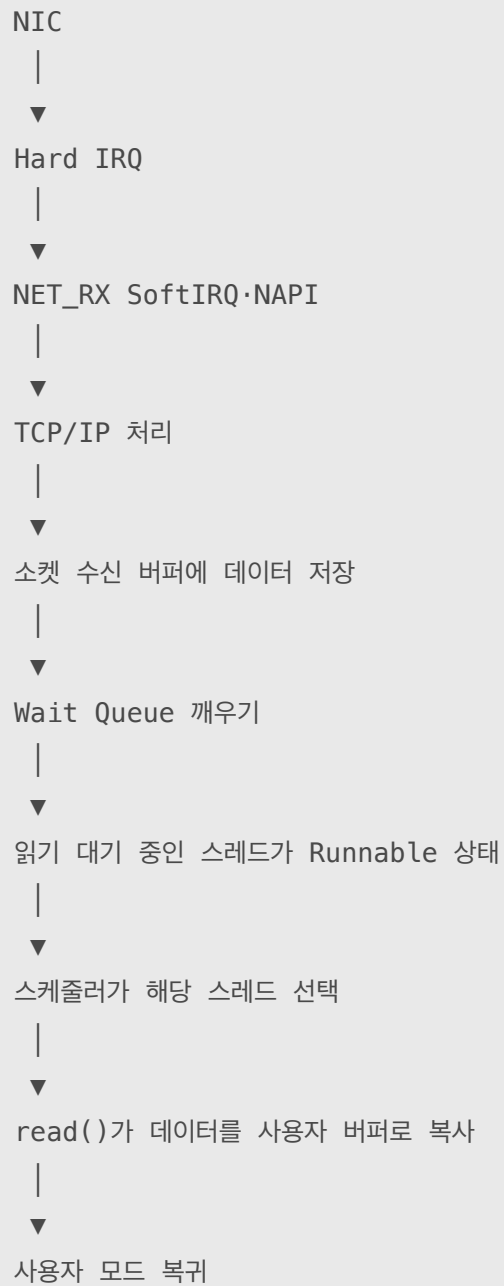
중요한 점은 다음과 같다.

Blocking은 CPU를 잡은 채 계속 확인하는 방식이 아니다. 스레드가 잠들어 CPU를 다른 실행 흐름에 넘기는 방식이다.

소켓 수신 버퍼가 비어 있다고 해보자.



이후 패킷이 도착한다.



[5] Non-blocking I/O

FD에 O_NONBLOCK을 설정하면 작업을 즉시 완료할 수 없을 때 스레드를 재우지 않고 바로 반환한다.

```
int flags = fcntl(fd, F_GETFL, 0);

fcntl(
    fd,
    F_SETFL,
    flags | O_NONBLOCK
);
```

소켓 수신 버퍼가 비어 있을 때 read()를 호출했다고 해보자.

```
ssize_t n = read(fd, buffer, size);
```

결과는 다음과 같다.

```
반환값: -1
errno: EAGAIN 또는 EWOULDBLOCK
```

```
non-blocking read()
    ↓
수신 버퍼 비어 있음
    ↓
즉시 EAGAIN 반환
```

하지만 프로그램이 다음처럼 계속 다시 호출하면 문제가 발생한다.

```
while (1) {
    ssize_t n = read(fd, buffer, size);

    if (n < 0 && errno == EAGAIN) {
        continue;
    }
}
```

결과적으로 CPU가 다음 작업을 반복한다.

```
read → EAGAIN
read → EAGAIN
read → EAGAIN
read → EAGAIN
```


이를 Busy Waiting이라고 한다.

따라서 Non-blocking FD는 일반적으로 select, poll, epoll 같은 I/O Multiplexing과 함께 사용한다. 준비됐다는 알림을 받은 뒤에만 read() 또는 write()를 시도한다.

[6] I/O Multiplexing

I/O Multiplexing은 하나의 실행 흐름이 여러 FD의 준비 상태를 한 번에 기다리는 방식이다.

FD 3: 클라이언트 A 소켓

FD 4: 클라이언트 B 소켓

FD 5: 클라이언트 C 소켓

FD 6: 리스닝 소켓



연결마다 OS 스레드를 하나씩 만들면 다음과 같은 비용이 발생할 수 있다.

- 많은 스레드 생성
- 스레드마다 스택 메모리 필요
- 문맥 교환 증가
- 스케줄링 부담 증가

I/O Multiplexing을 사용하면 적은 수의 이벤트 루프 스레드가 많은 소켓의 준비 상태를 관리할 수 있다. 다만 select, poll, epoll에는 공통점이 있다.

준비된 FD를 알려줄 뿐이며, 실제 데이터는 애플리케이션이 read(), recv(), write(), send()로 처리한다.

[7] select()

select()는 감시할 FD를 비트 집합인 fd_set에 담아 커널에 전달한다.

```
fd_set readfds;

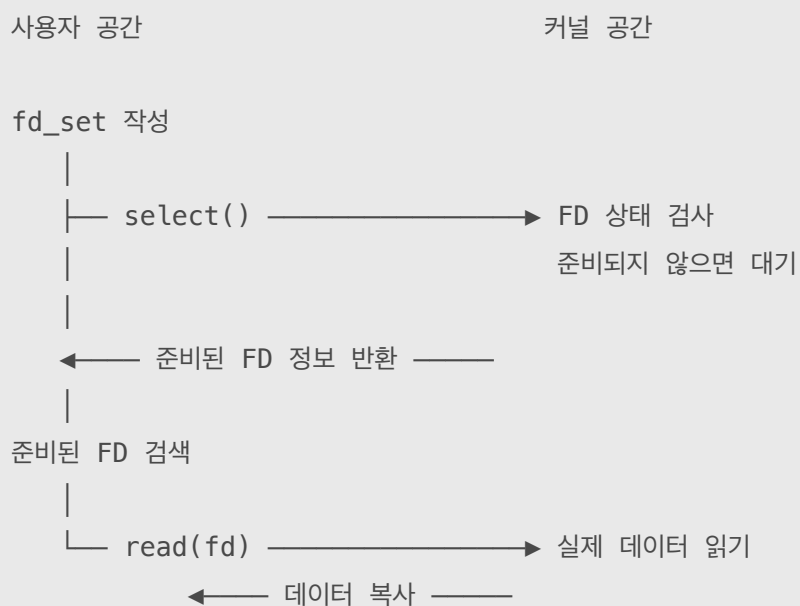
FD_ZERO(&readfds);

FD_SET(3, &readfds);
FD_SET(4, &readfds);

select(
    max_fd + 1,
    &readfds,
    NULL,
    NULL,
    NULL
);
```

[select()의 동작]

1. 사용자 프로그램이 감시할 FD를 fd_set에 표시한다.
2. select()를 호출한다.
3. fd_set이 사용자 공간에서 커널 공간으로 전달된다.
4. 커널이 0부터 nfd - 1 범위에서 FD 상태를 확인한다.
5. 준비된 FD가 없으면 호출한 스레드를 대기시킨다.
6. 이벤트가 발생해 깨어나면 감시 대상을 다시 확인한다.
7. 준비된 FD만 표시된 fd_set을 사용자 공간에 반환한다.
8. 사용자 프로그램이 fd_set을 검사해 준비된 FD를 찾는다.
9. 프로그램이 read() 또는 write()를 호출한다.



[select()의 한계]

- 호출할 때마다 fd_set을 커널로 전달해야 한다.
- 커널이 매번 감시 범위를 선형으로 확인한다.
- 사용자 프로그램도 준비된 FD를 찾기 위해 다시 검사한다.
- 일반적으로 FD_SETSIZE에 따른 1024개 제한이 있다.
- select()가 fd_set 내용을 변경하므로 다음 호출 전에 다시 만들어야 한다.

select()가 실제 데이터를 복사해 주는 것은 아니다.

```
select()  
→ 준비된 FD 정보 반환  
  
read()  
→ 실제 데이터 복사
```

[8] poll()

poll()은 select()의 비트 집합 대신 pollfd 구조체 배열을 사용한다.

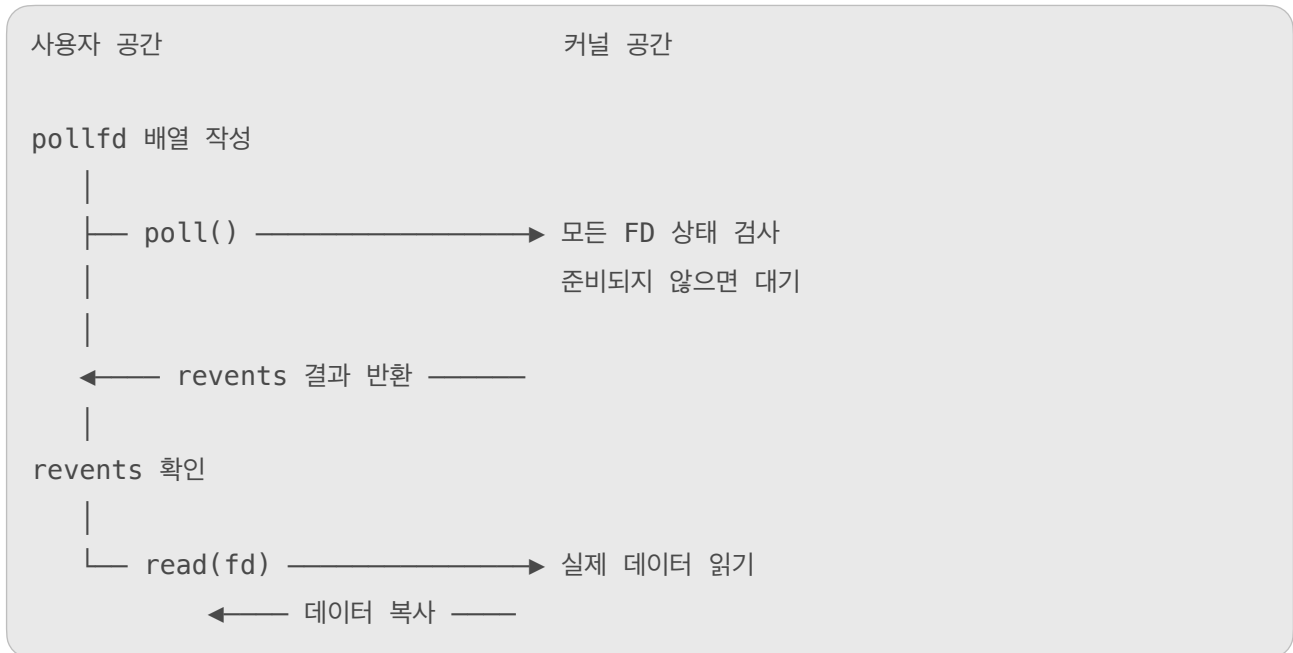
```
struct pollfd {  
    int    fd;        // 감시할 FD  
    short  events;     // 감시할 이벤트  
    short  revents;    // 실제 발생한 이벤트  
};
```

사용 예시는 다음과 같다.

```
struct pollfd fds[3];  
  
fds[0].fd = 3;  
fds[0].events = POLLIN;  
  
fds[1].fd = 4;  
fds[1].events = POLLIN;  
  
fds[2].fd = 5;  
fds[2].events = POLLIN;  
  
int count = poll(fds, 3, -1);
```

[poll()의 동작]

1. 사용자 프로그램이 pollfd 배열을 만든다.
2. 각 원소에 감시할 FD와 이벤트를 저장한다.
3. poll()을 호출한다.
4. 배열이 사용자 공간에서 커널 공간으로 전달된다.
5. 커널이 배열에 들어 있는 FD 상태를 확인한다.
6. 준비된 FD가 없으면 호출한 스레드를 대기시킨다.
7. 이벤트가 발생하면 해당 원소의 revents에 결과를 기록한다.
8. 결과가 담긴 배열을 사용자 공간으로 반환한다.
9. 사용자 프로그램이 배열을 순회하며 revents를 확인한다.
10. 준비된 FD에 read() 또는 write()를 호출한다.



poll()은 select()의 작은 FD 개수 제한을 피할 수 있다.

하지만 다음 문제는 남아 있다.

- 매번 pollfd 배열 전체를 전달한다.
- 커널이 매번 배열 전체를 검사한다.
- 사용자 프로그램도 배열을 순회한다.

감시 대상이 N개라면 확인 비용은 기본적으로 $O(N)$ 이다.

감시 FD 수: N

poll() 검사 비용: $O(N)$

[9] epoll의 핵심 구조

epoll은 감시할 FD 목록을 매번 전달하지 않는다.

감시 대상을 커널에 등록해 두고, 이벤트가 발생한 항목만 받아온다.

감시할 FD를 한 번 등록

↓

커널이 FD의 상태 변화 감지

↓

준비된 이벤트만 반환

예를 들어, FD 5, 7, 9를 등록했다고 해보자.

FD 5 → 준비되지 않음

FD 7 → 데이터 도착

FD 9 → 준비되지 않음

epoll_wait()는 FD 5, 7, 9를 모두 반환하지 않는다.

FD 7에 대응하는 이벤트만 반환한다.

epoll은 여러 FD를 커널에 등록해 두고,

그중 지금 I/O 준비가 된 FD에 대응하는 이벤트만 사용자 프로그램에 알려준다.

공간

커널 공간

epoll_create1()

→ epoll 객체 생성

epoll_ctl(ADD, FD 3, 4, 5)

→ 감시 목록에 등록

epoll_wait()

→ 이벤트가 없으면 대기

FD 4에 데이터 도착

Ready List에 FD 4 추가

← 이벤트가 발생한 FD 4 반환

read(FD 4)

→ 소켓 데이터 읽기

← 실제 데이터 복사

[10] epoll의 세 가지 시스템 콜

[10-1] [`epoll_create1()`: epoll 객체 생성]

```
int epfd = epoll_create1(EPOCH_CLOEXEC);
```

커널 내부에 여러 FD를 감시하기 위한 epoll 객체가 생성된다.

반환된 epfd는 이 객체를 가리키는 FD다.

```
epfd
  ↓
커널의 epoll 객체
```

epfd는 클라이언트 소켓이 아니다.

여러 클라이언트 FD의 감시를 관리하는 epoll 객체를 가리킨다.

[10-2] [`epoll_ctl()`: 감시 대상 관리]

```
epoll_ctl(
    epfd,
    EPOLL_CTL_ADD,
    client_fd,
    &event
);
```

지원하는 주요 명령은 다음과 같다.

`EPOLL_CTL_ADD`

→ 새로운 FD 등록

`EPOLL_CTL_MOD`

→ 등록된 FD의 감시 설정 변경

`EPOLL_CTL_DEL`

→ 감시 목록에서 FD 제거

[10-3] [`epoll_wait()`: 이벤트 대기]

커널 내부의 개념적인 흐름:

```
struct epoll_event events[128];

int count = epoll_wait(
    epfd,
    events,
    128,
    -1
);
```

준비된 이벤트가 없으면 호출한 스레드는 대기한다.

이벤트가 발생하면 준비된 이벤트를 `events` 배열로 반환한다.

```
128
→ 한 번에 반환받을 수 있는 최대 이벤트 개수

count
→ 실제로 반환된 이벤트 개수

-1
→ 시간 제한 없이 기다림
```

[11] `epoll_event`와 `event.data.fd`

클라이언트 FD 7에서 읽기 가능한 상태를 감시한다고 해보자.

```
int client_fd = 7;

struct epoll_event event;

event.events = EPOLLIN;
event.data.fd = client_fd;

epoll_ctl(
    epfd,
    EPOLL_CTL_ADD,
    client_fd,
    &event
);
```

위 코드에는 client_fd가 두 번 등장한다.

```
event.data.fd = client_fd;
```

```
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &event);
```

두 곳 모두 숫자 7을 전달하지만, 역할은 서로 다르다.

[11-1] [epoll_ctl()의 client_fd]

```
epoll_ctl(  
    epfd,  
    EPOLL_CTL_ADD,  
    client_fd,  
    &event  
);
```

여기에서 client_fd는 커널이 실제로 감시할 대상이다.

```
client_fd = 7
```

→ FD 7이 가리키는 소켓을 감시해라.

커널은 현재 프로세스의 FD Table에서 FD 7이 가리키는 소켓을 찾는다.

FD Table

FD 7 → 클라이언트 소켓

그리고 이 소켓이 EPOLLIN, 즉 읽을 수 있는 상태가 되는지 감시한다.

[11-2] [event.data.fd]

```
event.data.fd = client_fd;
```

event.data.fd는 감시 대사에 붙여 두는 이름표다.

감시 대상: FD 7이 가리키는 소켓

이름표: 숫자 7

epoll_wait()는 이벤트가 발생했을 때, 등록 과정에서 프로그램이 넣어 둔 data 값을 그대로 돌려준다. 따라서 프로그램은 반환된 이름표를 보고 어떤 건경르 처리해야 하는지 알 수 있다.

[11-3] [왜 FD 번호를 이름표로 저장하는가?]

epoll_wait()가 반환하는 epoll_event에는 실제 감시 대상의 FD 번호를 자동으로 넣어 주는 별도의 필드가 없다.

```
struct epoll_event {  
    uint32_t events;    // 어떤 이벤트가 발생했는가?  
    epoll_data_t data;  // 등록할 때 프로그램이 저장한 이름표  
};
```

따라서 프로그램은 등록할 때 data 영역에 FD 번호를 직접 저장하는 경우가 많다.

```
event.data.fd = client_fd;
```

이렇게 하면 이벤트가 발생했을 때 FD 번호를 다시 돌려받을 수 있다.

[11-4] [전체 동작 과정]

① 사용자 프로그램이 등록 정보를 만든다.

```
event.events = EPOLLIN;  
event.data.fd = 7;
```

감시 조건: 읽기 가능

이름표: 7

② 커널에 감시를 요청한다

```
epoll_ctl(epfd, EPOLL_CTL_ADD, 7, &event);
```

커널에는 개념적으로 다음 정보가 등록된다.

실제 감시 대상: FD 7이 가리키는 소켓

감시할 이벤트: EPOLLIN

함께 보관할 이름표: 7

③ FD 7의 소켓에 데이터가 도착한다.

FD 7이 가리키는 소켓 → 읽기 가능한 상태

④ `epoll_wait()`가 이벤트를 반환한다.

```
int count = epoll_wait(
    epfd,
    events,
    MAX_EVENTS,
    -1
);
```

커널은 사용자 프로그램이 등록했던 정보를 `events` 배열에 담아 돌려준다.

```
events[0].events = EPOLLIN
events[0].data.fd = 7
```

⑤ 프로그램이 이름표를 확인하고 소켓을 처리한다

```
for (int i = 0; i < count; i++) {
    int ready_fd = events[i].data.fd;

    recv(
        ready_fd,
        buffer,
        sizeof(buffer),
        0
    );
}
```

`events[i].data.fd`에 7이 들어 있으므로, 프로그램은 FD 7을 이용해 `recv()`를 호출한다.

[11-5] [감시 대상과 이름표의 차이]

코드	의미
<code>epoll_ctl(..., client_fd, ...)</code>	커널이 실제로 감시할 대상
<code>event.data.fd = client_fd</code>	이벤트가 발생하면 돌려받을 이름표

즉, 다음과 같이 이해하면 된다.

```
epoll_ctl()의 client_fd
→ “FD 7이 가리키는 소켓을 감시해줘.”

event.data.fd
→ “나중에 이벤트가 발생하면 숫자 7을 나에게 돌려줘.”
```

[11-6] [eventr.data.fd는 새로운 FD를 만드는 공간이 아니다]

```
event.data.fd = 7;
```

이 코드는 FD Table에 새로운 7번 항목을 만드는 것이 아니다.
FD 7은 이미 socket()이나 accept() 등의 시스템 콜을 통해 만들어진 상태다.

event.data.fd는 단지 숫자 7을 사용자 정의 데이터로 저장하는 것이다.

FD Table 등록

epoll 이름표 등록

FD 7 → 실제 소켓

data.fd = 7

커널의 열린 파일 관리 정보

이벤트 식별을 위한 사용자 데이터

[12] epoll 전체 흐름

[12-1] [감시 등록 및 대기]

먼저 감시할 FD와 이벤트 종류를 커널에 등록한다.

```
event.events = EPOLLIN;           // 읽기 가능한 상태를 감시
event.data.fd = client_fd;        // 나중에 돌려받을 식별값

epoll_ctl(
    epfd,
    EPOLL_CTL_ADD,
    client_fd,                     // 실제 감시 대상
    &event
);
```

의미는 다음과 같다.

“FD 7이 읽기 가능한 상태가 되는지 감시하고,
준비되면 식별값 7을 나에게 돌려줘.”

그리고 epoll_wait()로 이벤트를 기다린다.

```
int count = epoll_wait(epfd, events, MAX_EVENTS, -1);
```

[12-2] [실제 I/O 수행]

epoll_wait()가 식별값 7을 반환했다고 가정해보겠다.

```
int ready_fd = events[i].data.fd;
// ready_fd = 7
```

이것은 다음 의미이다.

“FD 7이 지금 I/O를 수행할 수 있는 상태입니다.”

프로그램은 FD 7을 커널에 다시 전달하여 실제 I/O를 요청한다.

읽기 이벤트였다면:

```
recv(ready_fd, buffer, sizeof(buffer), 0);
```

쓰기 이벤트였다면:

```
send(ready_fd, data, data_size, 0);
```

[12-3] [전체 흐름]

[감시 단계]

FD 7을 감시 대상으로 등록
EPOLLIN을 감시하도록 설정
이름표로 7을 저장

↓

epoll_wait()로 대기

↓

커널: “FD 7이 준비되었습니다.”

↓

[실제 I/O 단계]

사용자 프로그램이 FD 7을 확인

↓

recv(7, ...) 또는 send(7, ...) 호출

↓

커널이 FD Table에서 7번 항목을 찾음

↓

FD 7이 가리키는 소켓에서 실제 I/O 수행

[13] Interest List와 Ready List

epoll 객체는 개념적으로 감시 대상과 준비된 대상을 따로 관리한다.

[Interest List]

현재 I/O 준비가 된 FD에 대응하는 항목들이 연결된다.
FD 7만 읽기 가능한 상태라면 다음과 같다.

Ready List

└ FD 7에 대응하는 감시 항목

Ready List에는 실제 패킷 데이터가 들어가지 않는다.

Ready List

└ “FD 7을 지금 읽을 수 있다”는 이벤트 정보

FD 7의 소켓 수신 버퍼

└ 실제 네트워크 데이터

따라서 `epoll_wait()`가 이벤트를 반환한 이후에도 실제 데이터를 가져오려면 `recv()`를 호출해야 한다.

[13] 소켓 Wait Queue와 epoll 알림

소켓은 다음과 같은 정보를 관리한다.

소켓

└ 수신 버퍼

└ 송신 버퍼

└ 연결 상태

└ Wait Queue

`epoll_ctl()`로 소켓을 등록하면 epoll은 해당 소켓의 상태 변화 알림 경로와 연결된다.

이후 패킷이 도착하면 다음 과정이 진행된다.

패킷 도착

↓

네트워크 스택 처리

↓

소켓 수신 버퍼에 데이터 저장

↓

소켓 Wait Queue 알림

↓

epoll 콜백 실행

↓

Ready List에 감시 항목 연결

epoll_wait()가 대기 중이었다면 해당 스레드도 깨어난다.

Ready List에 항목 추가



epoll_wait() 스레드 깨움



준비된 이벤트를 events 배열로 반환

select, poll과의 가장 큰 차이는 다음과 같다.

epoll_wait() 를 호출할 때마다 전체 감시 목록을 다시 전달하지 않는다.

커널에 등록된 감시 관계를 유지하고 Ready List를 통해 준비된 이벤트를 받아온다.

소켓 상태 변경



epoll callback



├ Ready List에 항목 연결

└ epoll_wait 스레드 깨움



Runnable 상태



CPU에 스케줄됨



Ready List의 결과를 사용자 배열로 복사

[15] epoll_wait() 이후 실제 데이터 읽기

epoll_wait()는 실제 데이터를 가져오지 않는다.

준비된 이벤트와 등록할 때 저장한 식별값을 반환한다.

```
int count = epoll_wait(
    epfd,
    events,
    MAX_EVENTS,
    -1
);

for (int i = 0; i < count; i++) {
    int fd = events[i].data.fd;

    if (events[i].events & EPOLLIN) {
        ssize_t n = recv(
            fd,
            buffer,
            sizeof(buffer),
            0
        );
    }
}
```

역할을 구분하면 다음과 같다.

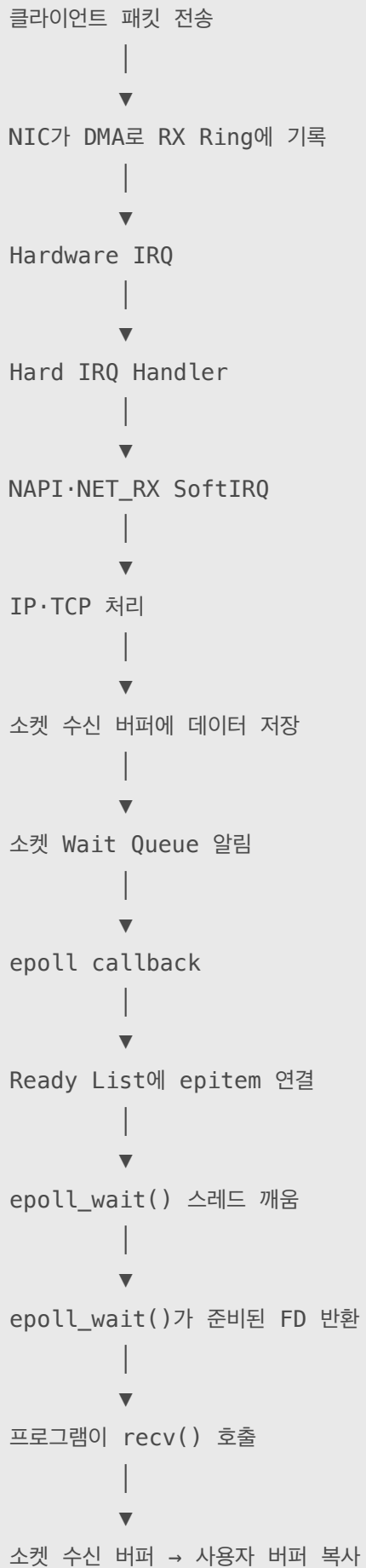
```
epoll_wait()
→ 준비된 FD에 대응하는 이벤트 반환

events[i].data.fd
→ 어떤 FD인지 식별

recv()
→ 실제 데이터 읽기

소켓 수신 버퍼
→ 사용자 버퍼로 데이터 복사
```

epoll_wait()는 “지금 읽을 수 있다”고 알려주고, recv()가 실제 데이터를 읽는다.



[16] Level-Triggered와 Edge-Triggered

[Level-Triggered(LT)]

LT는 epoll의 기본 방식이다.

FD가 계속 준비된 상태라면 다음 `epoll_wait()`에서도 다시 알려줄 수 있다.

소켓 수신 버퍼에 4KiB가 들어왔다고 해보자.

```
소켓 수신 버퍼
└─ 4KiB 데이터
```

프로그램이 1KiB만 읽었다.

```
recv(fd, buffer, 1024, 0);
```

수신 버퍼에 3KiB가 남아 있다.

```
4KiB 도착
  ↓
1KiB 읽음
  ↓
3KiB 남음
  ↓
다음 epoll_wait()에서도 EPOLLIN 가능
```

LT는 일부만 읽어도 다음 알림을 받을 수 있어 구현이 비교적 단순하다.

[Edge-Triggered(ET)]

ET는 다음과 같이 설정한다.

```
event.events = EPOLLIN | EPOLLET;
```

ET는 준비 상태의 변화가 발생했을 때 주로 알린다..

```
수신 버퍼 비어 있음
  ↓
데이터 도착
  ↓
수신 버퍼에 데이터 있음
  ↓
상태 변화가 발생했으므로 알림
```

4KiB가 들어왔는데 1KiB만 읽었다고 해보자.

수신 버퍼에 3KiB가 남아 있지만, 새로운 상태 변화가 발생하지 않으면 다시 알림을 받지 못할 수 있다.

따라서 ET에서는 일반적으로 다음 조건을 지켜야 한다.

Edge-Triggered

- Non-blocking FD 사용
- EAGAIN이 나올 때까지 반복해서 처리

```
while (1) {
    ssize_t n = recv(
        fd,
        buffer,
        sizeof(buffer),
        0
    );

    if (n > 0) {
        process(buffer, n);
        continue;
    }

    if (n == 0) {
        close_connection(fd);
        break;
    }

    if (errno == EAGAIN ||
        errno == EWOULDBLOCK) {
        break;
    }

    handle_error(fd);
    break;
}
```

[17] 리스닝 소켓과 accept()

서버의 리스닝 소켓이 EPOLLIN 상태라는 것은 일반적으로 Accept Queue에 처리할 새 연결이 있다는 뜻이다.

```
클라이언트 연결 요청
    ↓
TCP 연결 처리
    ↓
Accept Queue에 연결 저장
    ↓
리스닝 FD가 EPOLLIN
    ↓
epoll_wait() 반환
    ↓
accept4()
```

ET 방식에서는 Accept Queue에 준비된 연결을 가능한 한 모두 꺼내야 한다.
더 이상 연결이 없다는 EAGAIN이 반환되면 반복을 중단한다.

```
while (1) {
    int client = accept4(
        listen_fd,
        NULL,
        NULL,
        SOCK_NONBLOCK | SOCK_CLOEXEC
    );

    if (client >= 0) {
        add_to_epoll(client);
        continue;
    }

    if (errno == EAGAIN ||
        errno == EWOULDBLOCK) {
        break;
    }

    handle_accept_error();
    break;
}
```

[18] EPOLLOUT과 Partial Write

send()에 10KiB를 전달했다고 해서 항상 10KiB가 모두 커널 송신 버퍼에 들어가는 것은 아니다.

다음 상황을 생각해 보자.

전송하려는 데이터: 10KiB
송신 버퍼의 빈 공간: 4KiB

send()는 4096을 반환할 수 있다.

10KiB 전송 요청
├ 4KiB → 커널 송신 버퍼로 복사
└ 6KiB → 사용자 공간에 보관

나머지 6KiB는 애플리케이션이 보관해야 한다.

송신 버퍼에 공간이 생김
↓
EPOLLOUT
↓
남은 6KiB 다시 send()

따라서 연결별 상태에는 아직 보내지 못한 데이터를 보관할 송신 대기 버퍼가 필요할 수 있다.
남은 데이터를 전부 보냈다면 EPOLLOUT 감시를 해제해 불필요한 반복 알림을 막는다.

[19] 오류·종료와 EPOLLONESHOT

[오류와 연결 종료]

이벤트 루프는 다음 이벤트도 함께 확인해야 한다.

- EPOLLERR
- EPOLLHUP
- EPOLLRDHUP

EPOLLIN이 발생했더라도 read()가 0을 반환하면 상대가 정상적으로 송신을 종료한 상황일 수 있다.

[EPOLLONESHOT]

여러 Worker가 같은 epoll 인스턴스를 공유하면 동일한 연결을 동시에 처리하지 않도록 제어해야 할 수 있다.

EPOLLONESHOT을 사용하면 이벤트를 한 번 전달한 후 해당 FD를 비활성화한다.

```
event.events = EPOLLIN | EPOLLONESHOT;
```

처리가 끝난 뒤 EPOLL_CTL_MOD로 다시 활성화한다.

```
epoll_ctl(  
    epfd,  
    EPOLL_CTL_MOD,  
    fd,  
    &event  
);
```

FD에 이벤트 발생

↓

Worker에게 이벤트 전달

↓

FD 자동 비활성화

↓

Worker가 처리 완료

↓

EPOLL_CTL_MOD로 재활성화

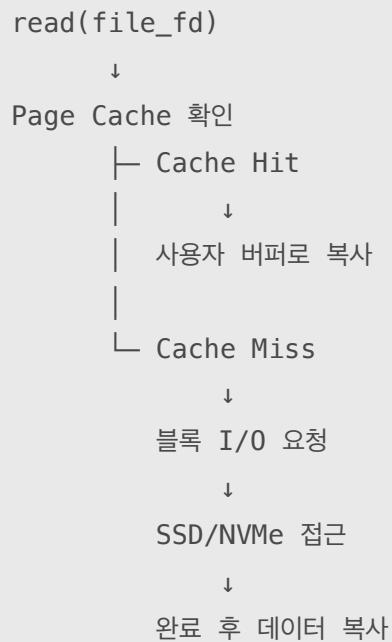
[20] 일반 파일과 epoll

epoll은 다음과 같은 FD에 주로 사용된다..

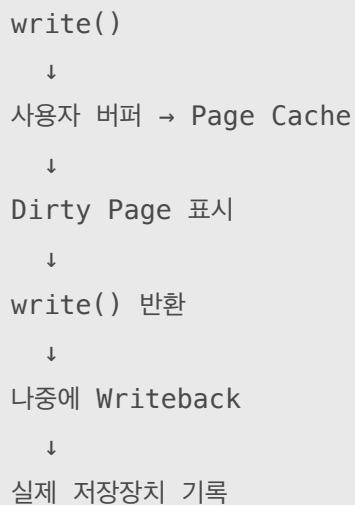
- 네트워크 소켓
- 파이프
- FIFO
- eventfd
- timerfd
- signalfd
- inotify
- 일부 장치 FD

일반 디스크 파일을 epoll_ctl()에 등록하면 EPERM이 발생할 수 있다.

일반 파일은 읽기 가능하다고 판단되더라도 Page Cache Miss가 발생하면 실제 블록 I/O 과정에서 스레드가 기다릴 수 있기 때문이다.

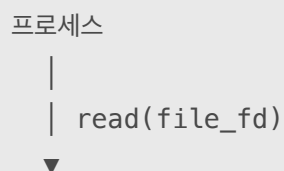


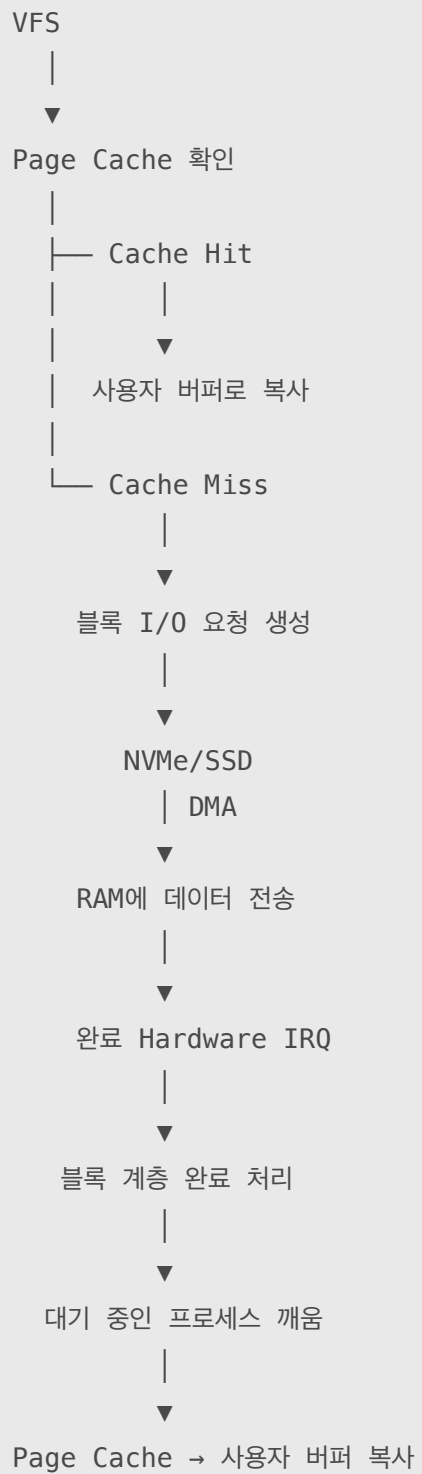
일반적인 버퍼드 write()도 데이터를 저장장치에 즉시 기록하지 않을 수 있다.



영구 저장 완료가 필요하면 상황에 따라 다음 함수를 사용한다.

```
fsync(fd);
fdatasync(fd);
```





[21] I/O 방식 최종 비교

방식	준비되지 않았을 때	여러 FD 감시	실제 I/O 수행
Blocking I/O	스레드가 잠들	보통 FD당 실행 흐름 필요	read/write
Non-blocking I/O	즉시 EAGAIN	자체만으로는 불편	read/write
select	준비된 FD까지 대기	전체 집합 반복 검사	별도 read/write
poll	준비된 FD까지 대기	배열 전체 반복 검사	별도 read/write
epoll	Ready List 이벤트까지 대기	준비된 FD 중심 처리	별도 read/write
io_uring	작업 제출 후 완료 수신	여러 작업 처리	커널에 I/O 작업 제출

[22] epoll은 비동기 I/O가 아니다

epoll은 Readiness Notification, 즉 준비 상태 알림이다.

```
epoll_wait(...);  
// FD 7을 지금 읽을 수 있다는 이벤트 반환  
  
recv(7, ...);  
// 애플리케이션이 실제 데이터를 읽음
```

반면 io_uring이나 AIO 계열은 애플리케이션이 I/O 작업을 제출하고, 나중에 커널로부터 작업 완료 결과를 받는 모델이다.

```
epoll  
→ I/O 준비 상태 알림  
→ 애플리케이션이 read/write 호출  
  
io_uring·AIO  
→ I/O 작업 제출  
→ 커널이 작업 수행  
→ 애플리케이션이 완료 결과 수신
```


[23] epoll 서버의 전체 이벤트 루프

지금까지 내용을 서버 흐름으로 합치면 다음과 같다.

1. 리스닝 소켓을 생성한다.
2. 리스닝 소켓을 Non-blocking으로 설정한다.
3. `epoll_create1()`로 epoll 인스턴스를 만든다.
4. 리스닝 소켓을 epoll에 등록한다.
5. `epoll_wait()`에서 준비된 이벤트를 기다린다.
6. 리스닝 소켓이면 `accept4()`를 반복한다.
7. EPOLLIN이면 `recv()`를 수행한다.
8. EPOLLOUT이면 남은 데이터를 `send()`한다.
9. 오류나 종료 이벤트가 발생하면 epoll에서 제거하고 FD를 닫는다.

```
while (1) {
    int count = epoll_wait(
        epfd,
        events,
        MAX_EVENTS,
        -1
    );

    for (int i = 0; i < count; i++) {
        int fd = events[i].data.fd;
        uint32_t ev = events[i].events;

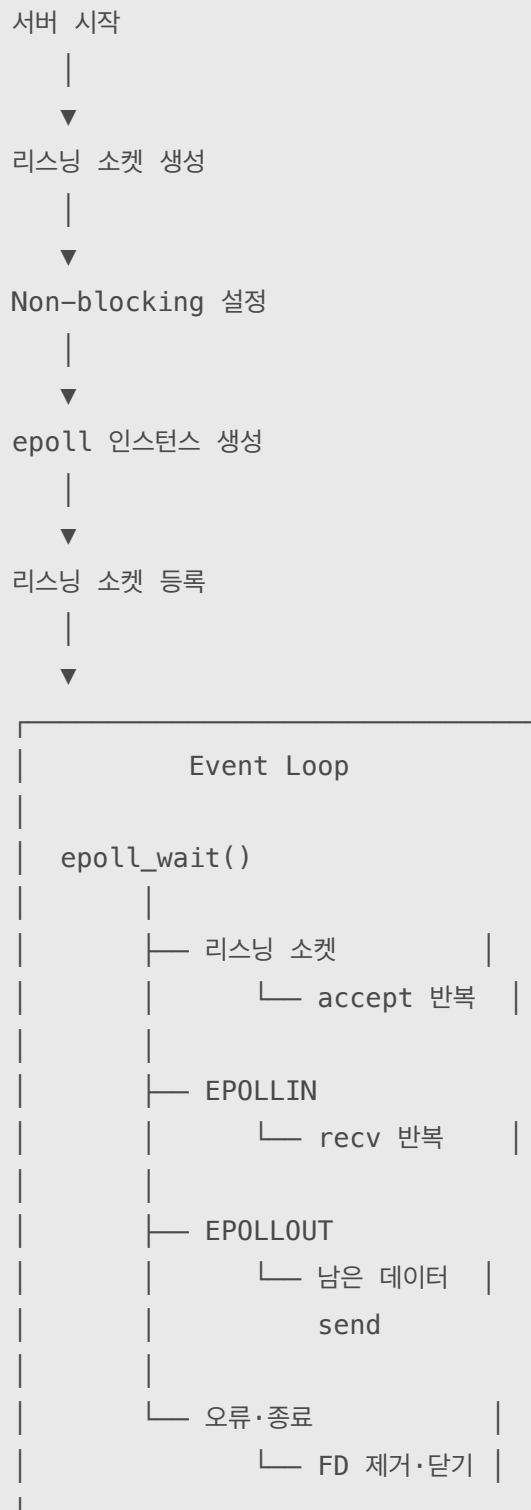
        if (fd == listen_fd) {
            accept_connections_until_eagain();
            continue;
        }

        if (ev & (EPOLLERR |
                  EPOLLHUP |
                  EPOLLRDHUP)) {
            close_connection(fd);
            continue;
        }

        if (ev & EPOLLIN) {
            receive_data_until_eagain(fd);
        }

        if (ev & EPOLLOUT) {
            send_pending_data_until_eagain(fd);
        }
    }
}
```

[24] epoll 서버의 전체 이벤트 루프



[25] Go 서버와 epoll

Go 코드에서 Accept()나 Read()는 Blocking I/O처럼 보인다.

```
connection, err := listener.Accept()

buffer := make([]byte, 4096)
n, err := connection.Read(buffer)
```

Linux에서 Go 런타임은 네트워크 소켓을 Non-blocking으로 관리하고 내부 netpoller에서 epoll을 사용할 수 있다.

읽을 데이터가 없다면 OS 스레드 전체가 아니라 해당 Goroutine을 대기시킨다.

```
Goroutine에서 Read()
    ↓
즉시 읽을 데이터 없음
    ↓
Goroutine Park
    ↓
소켓을 netpoller가 관리
    ↓
epoll이 준비 이벤트 감지
    ↓
Goroutine을 Runnable로 변경
    ↓
Go 스케줄러가 다시 실행
```

따라서 관점에 따라 다음과 같이 보인다.

```
사용자 코드
→ Blocking 방식처럼 간단하게 작성

Go 런타임 내부
→ Non-blocking socket
→ epoll
→ Goroutine 스케줄링
```

다만 일반 파일 I/O는 네트워크 소켓과 성격이 다르다.

일반 파일 I/O에서는 OS 스레드가 실제로 블로킹될 수 있다.

Go 런타임은 필요하면 다른 OS 스레드에서 다른 Goroutine이 계속 실행한다.

[26] 최종 비교

방식	감시 정보	준비된 FD 확인	실제 I/O
select	fd_set을 매번 전달	전체 범위 검사	read/write 별도 호출
poll	pollfd 배열을 매번 전달	배열 전체 검사	read/write 별도 호출
epoll	커널에 한 번 등록	Ready List 중심	read/write 별도 호출
io_uring	I/O 작업 제출	완료 결과 수신	커널이 제출된 작업 수행

- FD는 프로세스가 커널 I/O 객체를 가리키는 저수 번호다.
- 준비됨은 지금 I/O를 시도했을 때 블로킹되지 않고 어느 정도 진행할 수 있다는 뜻이다.
- Blocking I/O는 준비되지 않으면 스레드를 재운다.
- Non-blocking I/O는 준비되지 않으면 즉시 EAGAIN을 반환한다.
- select와 poll은 매 호출마다 전체 감시 정보를 전달하고 확인한다.
- epoll은 감시 대상을 커널에 등록하고 Ready List를 통해 준비된 이벤트를 반환한다.
- epoll_wait()가 실제 데이터를 읽어주는 것은 아니다.
- 실제 데이터 이동은 read(), recv(), write(), send()에서 일어난다.

epoll은 많은 FD 중에서 지금 I/O 가능한 FD를 효율적으로 알려주며, 실제 I/O는 애플리케이션이 직접 수행한다.